Michael Galileo Hasian (201344020)	**Rangkuman Minggu ke-1**	Senin, 29 Agustus 2022
		Aplikasi Mikrokontroler
3-TNK		Ferry Satria, BSEE, M.T. Rahmawati Hasanah, S.ST.,M.T.

Perbedaan Mikrokontroler dan Mikroprosessor	
Mikroprosesor	**Mikrokontroler**
- CPU - Register - BUS (Data Bus, Address Bus, Kontrol Bus)	- CPU - Register - BUS (Data Bus, Address Bus, Kontrol Bus) - Unit memori - I/O port - ADC, DAC, Timer - Interrupt Controler - RAM

1. Perintah Operasi ADD

   Perintah (Mnemonic) ADD merupakan perintah proses penjumlahan dua register tanpa nilai carry. Hasilnya disimpan pada 1 area register

   Penulisan Syntax :

   ADD Rd,Rr

   Keterangan :

   Rd merupakan Register yang menyimpan data yang akan dijumlahkan, Rr merupakan Register yang menyimpan nilai penjumlahnya

   Operands (Register yg digunakan) :

   $0 \leq d \leq 31, 0 \leq r \leq 31$

2. Perintah Logic AND dan Immediate

   Perintah (Mnemonic) AND berfungsi untuk melakukan operasi logika AND antar register yang ada.

Penulisan Syntax :

AND Rd,Rr

Keterangan :

Rd merupakan Register yang menyimpan data yang telah dilakukan operasi Rd.Rr

Operands (Register yg digunakan) :

$0 \leq d \leq 31, 0 \leq r \leq 31$

Pada beberapa perintah memiliki varian Immediate, yaitu data yang masuk akan langsung diarahkan ke instruksi. Biasanya perintah ynag memiliki varian Immediate akan memiliki akhiran 'I' pada perintahnya.Contohnya ANDI yang berarti perintah dasarnya adalah AND namun memiliki fungsi Immediate.

Ciri-ciri perintah dengan fungsi Immediate :

➢ pada perintah yang menggunakan fungsi Immediate disarankan untuk hanya menggunakan register R16 - R31. Hal ini disebabkan oleh seringnya terjadi kesalahan (error) saat menggunakan register dibawah R16 (R0 – R15)

➢ Operand kedunya ialah constant (K) yang bernilai 8 bit.

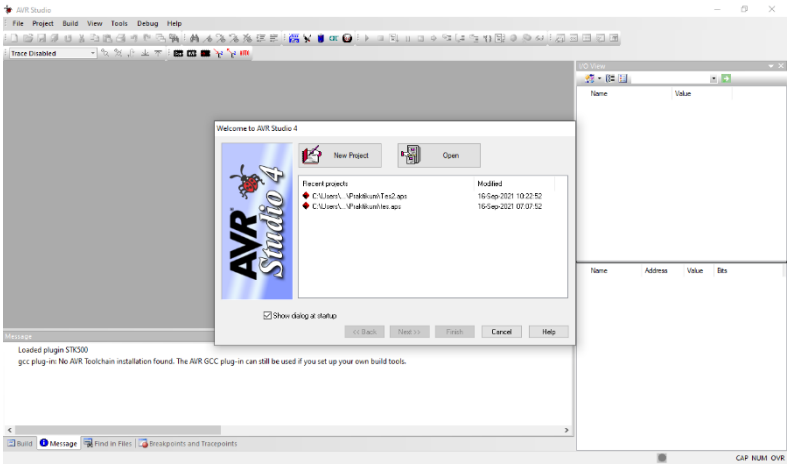
Range K = 0 – 255

3. Memulai Pemrograman Mikrokontroller

Dalam percobaannya menggunakan bantuan aplikasi AVR Studio 4

1. Cara Membuat Projek Baru

1) Buka aplikasi "AVR Studio 4"

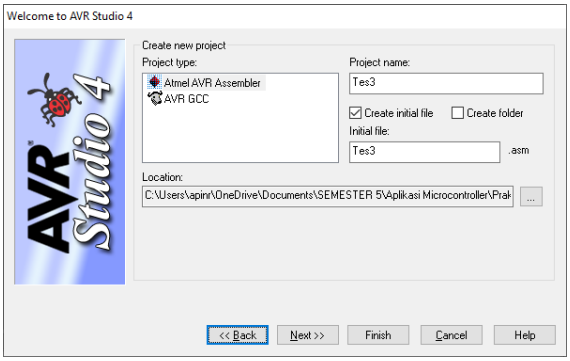


Tampilan utama dari **AVR Studio 4**

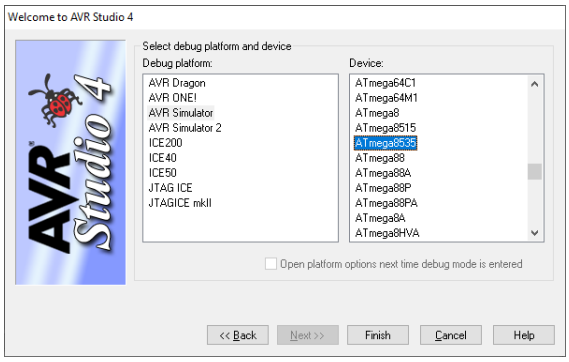
2) Klik "New Project"

3) Pilih "Atmel AVR Assembler" pada Project Type, lalu beri nama pada kolom "Project

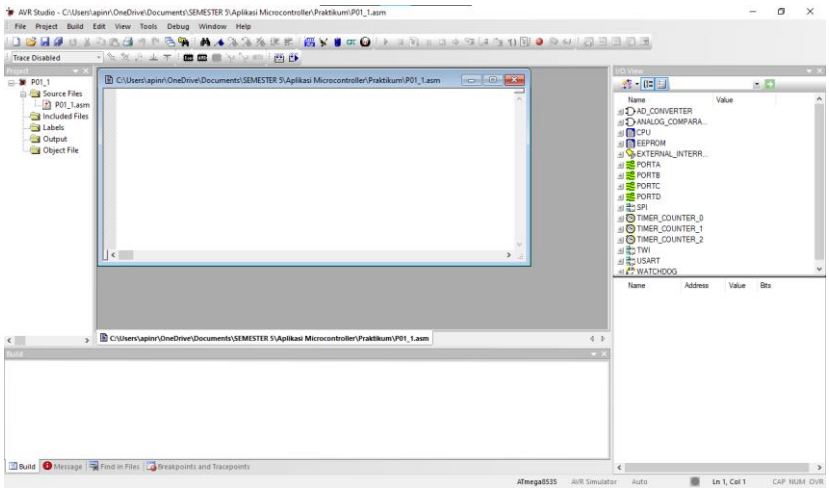
Name”



4) Pilih “AVR Simulator”, lalu pilih “ATmega 8535”, lalu klik tombol finish



5) Bila sudah maka akan tampil editor seperti gambar berikut :



4. Contoh Program Sederhana

4.1. Program Sederhana Pertama

```
.include "m8535def.inc"
.org 0x00
rjmp main
main :
```

```
ldi r16,low(ramend)
out spl,r16                    ; spl = stack pointer low
ldi r16,high(ramend)
out sph,r16

ldi r17,0
akhir: rjmp akhir
```

Penjelasan :

- Baris pertama merupakan header dengan yang berada dalam tanda petik ganda merupakan library, baris wajib
- Baris kedua memerintahkan supaya program ditulis mulai alamat memori 0
- Baris ketiga, rjmp : relative jump ke label main
- Baris keempat digunakan untuk mengambil alamat terakhir dari low ramend menuju r16. Ramend merupakan alamat paling akhir dari suatu RAM
- Baris kelima digunakan untuk memasukkan nilai dari r16 ke spl
- Baris keenam dan ketujuh sama seperti baris keempat dan kelima hanya saja untuk high ramend
- Baris 4-7 merupakan inisialisasi stack, baris wajib
- Baris terakhir digunakan untuk menyetop program
- Perintah LDI hanya bisa digunakan untuk r16 – r31

4.2. Program Sederhana Kedua (Penjumlahan)

```
.include "m8535def.inc"
.org 0x00
rjmp main
main :
ldi r16,low(ramend)
out spl,r16                ; spl = stack pointer low
ldi r16,high(ramend)
out sph,r16

ldi r16,0x5D
```

```
ldi r17,0xA7
ldi r18,0x04    ;Input data pertama

ldi r19,0xD8
ldi r20,0xAC
ldi r21,0x0F    ;Input data kedua

add r24,r16
add r24,r19
adc r25,r17
adc r25,r20
adc r26,r18
adc r26,r21

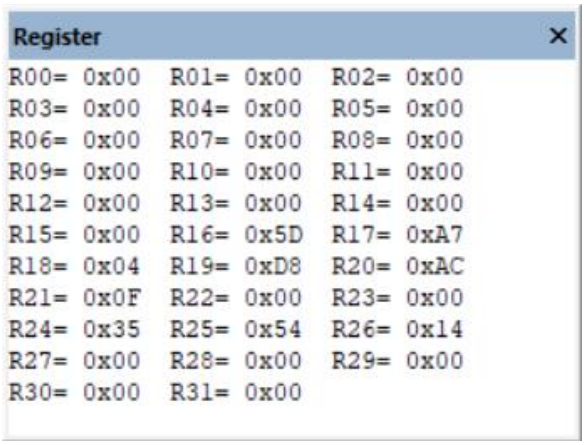
akhir: rjmp akhir
```

Penjelasan

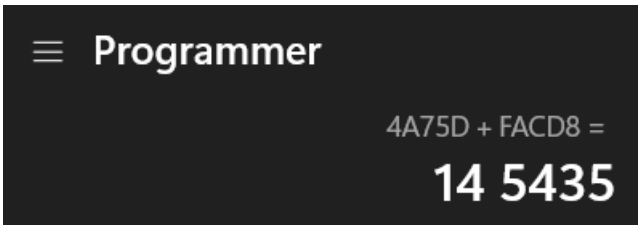
- Baris pertama merupakan header dengan yang berada dalam tanda petik ganda merupakan library, baris wajib
- Baris kedua memerintahkan supaya program ditulis mulai alamat memori 0
- Baris ketiga, rjmp : relative jump ke label main
- Baris keempat digunakan untuk mengambil alamat terakhir dari low ramend menuju r16. Ramend merupakan alamat paling akhir dari suatu RAM
- Baris kelima digunakan untuk memasukkan nilai dari r16 ke spl
- Baris keenam dan ketujuh sama seperti baris keempat dan kelima hanya saja untuk high ramend
- Baris 4-7 merupakan inisialisasi stack, baris wajib
- Baris terakhir digunakan untuk menyetop program
- Program ini bertujuan untuk melakukan penjumlahan 2 bilangan heksadesimal 6 digit yang terdapat pada register r16 s/d r18 untuk data pertama dan r19 s/d r21 untuk data kedua.
- Hasil penjumlahan kedua data disimpan di register r24 s/d r26.

- Proses penjumlahan dengan menggunakan perintah ‘add’ terdapat 2 tahap, di mana pada tahap pertama adalah memberikan nilai ke suatu register menggunakan perintah “ldi” dan tahap kedua menambahkan kedua register yang telah diberi nilai pada tahap pertama menggunakan perintah “add”.
- Jika terdapat carry pada proses penjumlahan sebelumnya maka untuk menambahkan carry dalam proses penjumlahan dapat menggunakan perintah “adc”.

Pembuktian



Nilai pada register R26 = 0x14, R25 = 0x54, dan R24 = 0x35



4.3. Program Sederhana Ketiga (Pengurangan)

```
.include "m8535def.inc"
.org 0x00
rjmp main
main :
ldi r16,low(ramend)
out spl,r16                ; spl = stack pointer low
ldi r16,high(ramend)
out sph,r16

ldi r16,0xC2
```

```
ldi r17,0xAD
ldi r18,0x75    ;Input data pertama

ldi r29,0xE9
ldi r30,0xDC
ldi r31,0x6F    ;Input data kedua

sub r16,r29
sbc r17,r30
sbc r18,r31

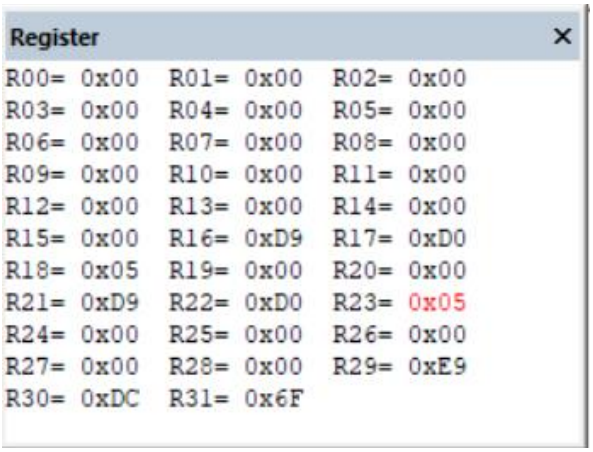
mov r21,r16
mov r22,r17
mov r23,r18
akhir: rjmp akhir
```

Penjelasan :

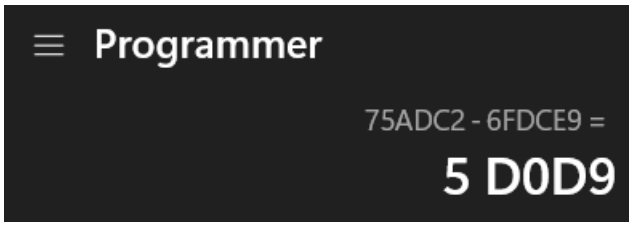
- Baris pertama merupakan header dengan yang berada dalam tanda petik ganda merupakan library, baris wajib
- Baris kedua memerintahkan supaya program ditulis mulai alamat memori 0
- Baris ketiga, rjmp : relative jump ke label main
- Baris keempat digunakan untuk mengambil alamat terakhir dari low ramend menuju r16. Ramend merupakan alamat paling akhir dari suatu RAM
- Baris kelima digunakan untuk memasukkan nilai dari r16 ke spl
- Baris keenam dan ketujuh sama seperti baris keempat dan kelima hanya saja untuk high ramend
- Baris 4-7 merupakan inisialisasi stack (Baris wajib)
- Baris terakhir digunakan untuk menyetop program
- Program ini bertujuan untuk melakukan pengurangan 2 bilangan heksadesimal 6 digit yang terdapat pada register r16 s/d r18 untuk data pertama dan r29 s/d r31 untuk data kedua.
- Hasil penjumlahan kedua data disimpan di register r21 s/d r23.

- Proses pengurangan dengan menggunakan perintah “sub” terdapat 2 tahap, di mana pada tahap pertama adalah memberikan nilai ke suatu register menggunakan perintah “ldi” dan tahap kedua mengurangi kedua register yang telah diberi nilai pada tahap pertama menggunakan perintah “sub”.
- Jika terdapat borrow pada proses pengurangan sebelumnya maka untuk menambahkan borrow dalam proses pengurangan dapat menggunakan perintah “sbc”.

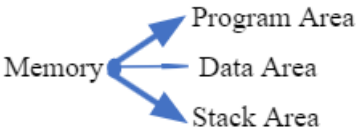
Pembuktian



Nilai pada register R21 = 0xD9, R22 = 0xD0, dan R23 = 0x05



5. Register dan Memori di ATMega 8535



Terdapat 2 jenis perintah :

1. Ditujukan untuk MC
   - Berupa set instruksi
2. Ditujukan untuk Assembler
   - Assembler digunakan untuk mengubah bahasa assembly menjadi bahasa mesin
   - Berupa Assembler Directive (Diawali dengan titik pada awal perintah/dot command)

Contoh Dot Command :

.include “m8535def.inc”

.org 0x00

Pointer Register

Terdapat 3 jenis Pointer Register di ATMega 8535, yaitu pointer X, pointer Y, dan pointer Z. Dari ketiga pointer tersebut, Pointer Z merupakan yang paling istimewa, karena Pointer Z dapat digunakan di ketiga Area pada memory. Sedangkan 2 pointer lainnya tidak dapat menggunakan Program Area.

1. X = Menunjuk kepada data area, merupakan R27 dan R26
2. Y = Menunjuk kepada data area, merupakan R29 dan R28
3. Z = Menunjuk kepada data area dan program area, merupakan 2 register terakhir (Zh = R31, Zl = R30)

Contoh Program Sederhana melibatkan Pointer Register

```
.include "m8535def.inc"
.org 0x00
rjmp main
main :
ldi r16,low(ramend)
out spl,r16                    ; spl = stack pointer low
ldi r16,high(ramend)
out sph,r16

ldi zl,low(2*BIL)
ldi zh,high(2*BIL)

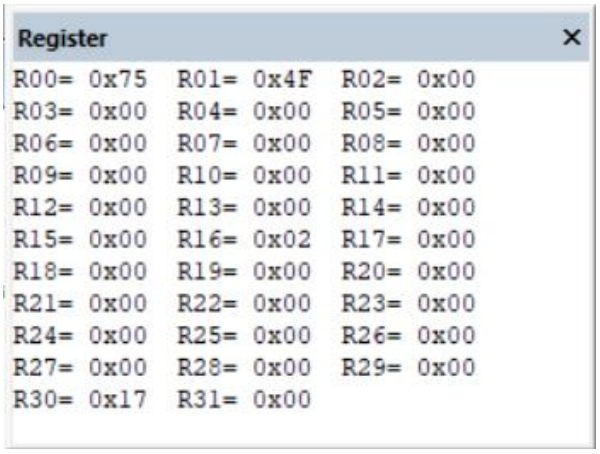
lpm r0,Z
inc r30
lpm r1,Z
akhir: rjmp akhir

BIL:
.db 0x75,0x4f
```

Penjelasan :

- Program ini bertujuan untuk memindahkan data dari program area yang tersimpan pada Label BIL.
- Perintah **ldi zl,low(2*BIL)** dan **ldi zh,high(2*BIL)** berfungsi untuk mengambil alamat dari label BIL.
- Perintah **lpm r0,Z** dan **lpm r1,Z** berfungsi untuk memindahkan data dari register Z yang sudah dialamatkan label BIL ke register tujuan, dalam hal ini r0 dan r1.
- Perintah **inc r30** berfungsi untuk menaikkan alamat dari alamat register Z yang diakses.

Pembuktian



Nilai pada register R0 = 0x75 dan R1 = 0x4F

```
.include "m8535def.inc"
.org 0x00
rjmp main

main :
ldi r16,low(ramend)
out spl,r16                    ; spl = stack pointer low
ldi r16,high(ramend)
out sph,r16

ldi r28,0x60
```

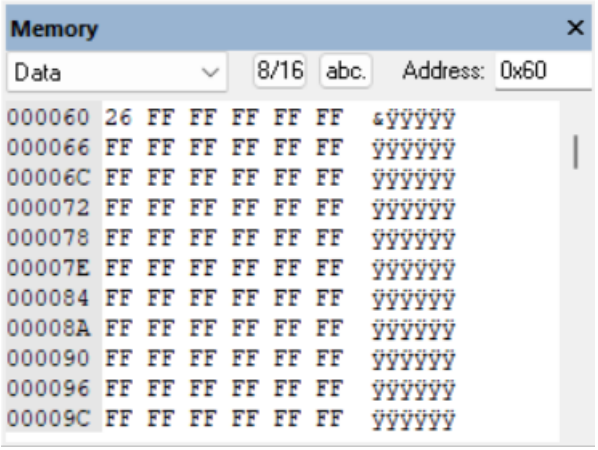
ld r2,y+

ld r3,y

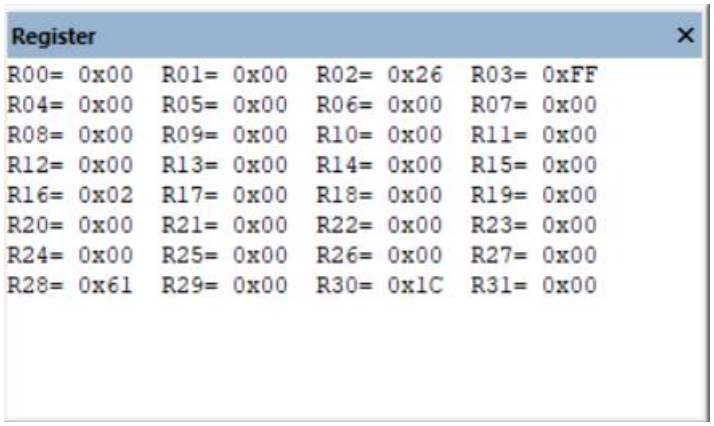
Penjelasan :

- Program ini bertujuan untuk memindahkan data dari data area menggunakan register pointer Y.
- Untuk memindahkan data dari data area menuju suatu register maka nilai pada data area perlu diisi terlebih dahulu.
- Perintah **ldi r28,0x60** berfungsi untuk mengambil alamat awal dari data area dan memindahkannya ke r28.
- Perintah **ld r2,y+** dan **ld r3,y** berfungsi untuk memindahkan data dari register pointer Y yang sudah dialamatkan data area ke register tujuan, dalam hal ini r2 dan r3.
- Lambang "+" pada akhir perintah **ld r2,y+** menandakan bahwa nilai dari register pointer Y bertambah setiap register pointer Y dipanggil.

Pembuktian



Nilai pada data area alamat pertama sebesar 26H dan FFH



Nilai pada register R2 = 0x26 dan R3 = 0xFF